

Design and Development Shop and pay Android app for a supermarket

The Acme Electronic Supermarket

Henrique Manuel Cordeiro Rodrigues Gardé
up202108725

João Tomás Matos Fernandes Garcia Padrão
up202108766

José Francisco Reis Pedreiras Neves Veiga
up202108753



Master's in Informatics and Computer Engineering

Mobile Computing

2024/2025

Contents

1	Introduction	2
2	System Overview	2
3	Android Architecture	4
3.1	Domain Layer	4
3.2	Data Layer	5
3.3	Presentation Layer and MVVM	6
3.4	Dependency injection with Hilt	7
4	Server architecture	7
4.1	Layered Structure	7
4.2	Security and Cryptography	8
4.3	Database Schema	9
4.4	Persistence and Data Integrity	10
4.5	API Structure and Communication	10
4.6	Dependency Injection and Modularity	11
5	Features & Technologies	11
6	Conclusions	12

1 Introduction

This paper presents an Android-based secure shopping and payment system called **The Acme Electronic Supermarket**. The system consists of two Android applications - the **Client app** and the **Terminal Checkout App** - as well as a central **Server**.

Both Android apps are developed using modern technologies such as **Kotlin** and **Jetpack Compose**, alongside many state-of-the-art libraries, which will be discussed in detail in the following chapters.

The Server is implemented as a **RESTful API** using Spring Boot and Kotlin, making our entire codebase written with this language.

Some of the high-level functionalities of our system are:

- User registration via the Server and local login/logout;
- QR code generation and scanning to manage products and transactions;
- Focus on cryptography techniques to ensure secure communication between all three components;
- Local storage of user data, transaction history and vouchers, with optional synchronization to the Server on demand.

In the upcoming chapters, we will explore the system architecture of both Android apps and the Server, and explain the key design and implementation decisions, technologies and libraries used and expand on the main features outlined above.

2 System Overview

As mentioned above, the Supermarket System is composed of three main components:

- A **Client App** used by customers for scanning products from the Supermarket, managing accounts, and completing purchases.
- A **Terminal Checkout App** used by store attendants or integrated in machines to scan the QR Code transaction generated in the Client App and handle checkouts.
- A **central Server**, which acts as the backend system, exposing RESTful APIs to manage users, transactions and vouchers logic.

The interactions between these components are illustrated in the figure below.

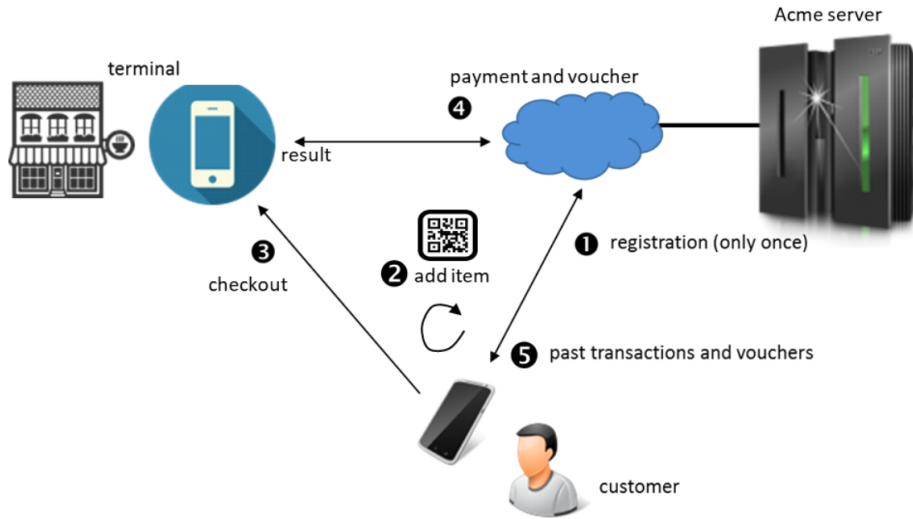


Figure 1: System Overview Diagram

1. **User Registration:** The User signs up on the Client App, which communicates with the Server to create a new account. The login is local in the Client App, as the User's credentials is stored in the device.
2. **Adding items to cart:** Products in the store are tagged with QR codes that uniquely identify them. Customers launch the camera in the Client App and scan these QR codes, decoding the useful information and locally building a cart composed of a list of products.
3. **Checkout:** Once ready to pay, the customer generates a QR code in the app and presents it to the Terminal Checkout App. The QR code is scanned and the data extracted and ready for a request to the server.
4. **Payment and Voucher processing:** The terminal sends the checkout data mentioned above to the Server. The Server verifies the purchase, creates a voucher if applicable and applies the available discount of the user if selected. It sends back to the Terminal the final price, discount used and if the transaction was successful.
5. **Viewing past transactions and vouchers:** Customers can request their transaction history or available vouchers at any time. This information is fetched from the Server Database and stored on the customer's device, so it will be persisted and displayed every time they open the app.

All interactions between the components followed a structured DTO-based design and are secure using cryptography techniques; these topics will be discussed further on the next sections.

3 Android Architecture

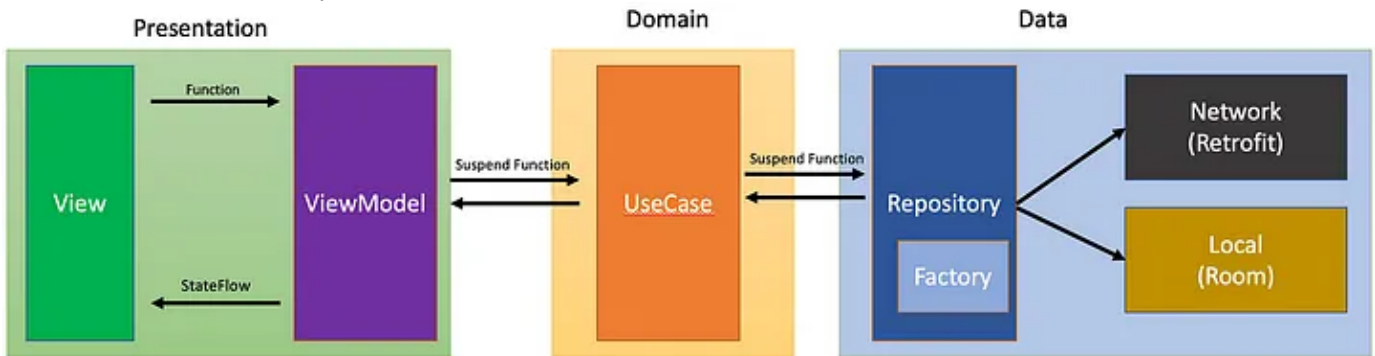
In this section, we mainly focus on the architecture of the **Client Android App**. The Terminal App also follows this architecture, but in a much simpler fashion, as the app only has one purpose: to scan QR codes, send them to the Server and show the Server's response, either success or failure.

The main pillars and inspirations of our architecture are: **Clean Architecture**, **MVVM**: Model-View-ViewModel and **Dependency Injection**.

Clean Architecture: A Craftsman's Guide to Software Structure and Design [1] is a book written by the legendary Robert Cecil Martin, commonly known as "Uncle Bob" in the Software Engineering community. The book emphasizes essential principles of software design such as **separation of concerns**, **dependency inversion** and **testability**. It presents a layered architectural approach where the core business logic is kept independent of external frameworks, databases, and user interfaces. This makes systems easier to maintain, test, and evolve over time. Clean Architecture has become a foundational reference for developers aiming to write robust, scalable, and flexible applications.

More specifically, we have followed the **Clean Architecture Manifest** [2] GitHub repository, which builds upon the concepts from the book and adapts them specifically for Android development using Kotlin.

The architecture is composed of 3 main layers: **presentation**, **domain** and **data** layers.



3.1 Domain Layer

The **domain layer** serves as the core of our application architecture. It encapsulates the business logic and remains completely independent of both the presentation and data layers. This strict separation of concerns ensures that the domain layer remains stable and unaffected by changes in external systems such as the UI or data sources.

Additionally, it's worth noting that there is no direct communication between the data and presentation layers. All interactions are mediated through the domain layer, promoting a clean, testable, and maintainable architecture.

This layer contains the following folders:

- **model:** contains all the data classes used in memory within the app. These models represent the core domain entities and are independent of any external data representation.
- **repository:** defines abstract interfaces that specify the contracts for data operations. These interfaces are implemented in the data layer, allowing the domain layer to remain decoupled from the actual data sources.
- **usecases:** encapsulates specific units of business logic. Each use case represents one or a small number of well-defined operations that the app can perform and are invoked by the ViewModel in the presentation layer.
- **local:** contains a single UserDataStore interface that provides access to user session and preference data
- **crypto:** contains the CryptoManager interface, which defines all the functions required to perform cryptographic operations, such as key generation, encryption and decryption. It also abstracts access to the Android KeyStore, which ensure that users' private keys are securely stored and managed in a platform-safe manner.

3.2 Data Layer

The **data layer** is responsible for implementing the interfaces defined in the domain layer and handling all data-related operations. It is the bridge between the domain logic and external data sources such as local databases, remote APIs, or platform services. This layer provides concrete implementations of repositories, manages data sources and coordinates transformation and persistence logic - all while keeping the domain layer completely unaware of where or how the data is retrieved or stored.

This layer is organized into the following components:

- **remote:** defines the interfaces responsible for network communication with external services. In this case, there is a single **SuperMarketApi** interface that declares endpoints using Retrofit to handle the communication with the Server. These functions are asynchronous and rely on secure mechanisms, including nonces and digital signatures, to ensure request authenticity.
- **local:** manages interaction with local storage systems such as Room databases and Jetpack DataStore. It also contains the implementation of the platform-specific abstraction **UserDataStore**.
- **dto:** this folder contains all the DTOs (data transfer objects) classes used to represent the structure of data received from or sent to remote APIs. These classes should match the API contract exactly and may differ from the domain models used internally. DTOs are never exposed outside the

data layer, ensuring a clear separation between external data and internal business logic.

- **mapper**: this folder includes utility functions responsible for converting between DTOs, domain and entity (used for Room databases) folders. Mappers ensure that the domain layer works exclusively with clean, consistent and API-agnostic data structures. If the structure of a remote API changes, we only need to update the DTOs and their mappers. The business logic remains completely unaffected, preserving its stability and independence.
- **crypto**: implements the **CryptoManager** interface defined in the domain layer. This folder contains the concrete logic for cryptographic operations such as key generation, encryption, decryption, and secure storage using the Android KeyStore. By keeping the implementation here and depending only on the interface in the domain layer, we maintain a clear separation of concerns and ensure that cryptographic logic can be tested, replaced, or extended without impacting the core business logic.

In our implementation, the **ViewModel** in the presentation layer communicates with the domain layer through well-defined **use cases**. These use cases expose the business functionality needed by the UI without revealing internal details of the domain logic. Meanwhile, the domain layer depends on abstract **repository interfaces** to access data. These interfaces are declared within the domain layer but implemented in the data layer. This approach adheres to the Dependency Inversion Principle and guarantees that the domain layer has no direct knowledge of the data layer's implementation details.

3.3 Presentation Layer and MVVM

The **presentation layer** is responsible for displaying data to the user and handling user interactions. It is built using the **Model-View-ViewModel (MVVM)** architectural pattern, which promotes a clear separation of concerns between UI logic and business logic.

In this layer, each screen is represented by a composable function located in the **screens** folder. Every screen has its own dedicated **ViewModel**, which communicates with the domain layer through use cases. The **ViewModel** maintains a **UI state** data class that holds all the observable properties needed to render the UI in a reactive and lifecycle-aware way.

Navigation between screens is handled via the **navigation** folder, which defines a centralized navigation graph. This setup ensures that routing logic remains decoupled from individual screens.

By following MVVM and structuring the presentation layer in this way, we ensure the UI components remain as stateless and declarative as possible, while business and state management logic stays within the **ViewModel**.

3.4 Dependency injection with Hilt

To support the modular structure and strict separation of concerns in our architecture, we rely on **Hilt** for dependency injection. Without Hilt, it would be harder to achieve the same level of flexibility, testability, and decoupling between layers.

Hilt allows us to inject dependencies at compile-time, ensuring that all components - from use cases to repositories and platform services - are provided when needed without manual wiring. For example, in a use case, we can request any repository it needs simply by adding it as a constructor parameter. Thanks to Hilt's compile-time graph generation, this does not break anything nor require manual instantiation, maintaining both code cleanliness and maintainability.

We also use *hiltViewModel()* in our presentation layer, which enables seamless injection of dependencies into ViewModels. This ensures that ViewModels are configured and scoped correctly to their lifecycle, with all required use cases and services injected automatically.

By using Hilt, we avoid tight coupling, eliminate boilerplate, and gain the flexibility to scale or refactor individual modules without disrupting the rest of the application. We define separate modules in the **di** folder, such as **NetworkModule** for providing Retrofit and Api-related dependencies and **RepositoryModule** to map domain-level repository interfaces to their concrete implementations in the data layer.

4 Server architecture

The Server component of the Acme Electronic Supermarket system is developed using Spring Boot and Kotlin, following a modular and scalable architecture. It serves as the central backend responsible for handling user accounts, transactions, vouchers, cryptographic operations, and API communication with both Android applications.

Our architecture is inspired by Domain-Driven Design (DDD) principles, which promote separation of concerns, encapsulation of business logic, and the use of clearly defined layers. The project is divided into distinct packages to isolate domain logic, business rules, and infrastructural concerns, improving maintainability and testability.

4.1 Layered Structure

The Server codebase is organized into the following main layers:

- **Entities Layer:** Contains domain models representing core business concepts such as `User`, `Product`, `Transaction`, `TransactionProduct` and `Voucher`. These classes are annotated with JPA annotations and map directly to database tables.

- **Business Layer:** Implements the core logic of the application. This layer includes services and interfaces that encapsulate business rules such as calculating discounts, issuing vouchers, seeing past transactions and unused vouchers, and performing validations. Each service is focused on a single responsibility to ensure clean and modular design.
- **Controller Layer:** Exposes the RESTful API endpoints using Spring's `@RestController` annotations. These controllers handle HTTP requests from the Android applications and delegate processing to the corresponding services. All communication follows a structured DTO-based approach to ensure a clear contract between clients and the server.
- **DTO Layer:** Defines data transfer objects used to carry data between the server and clients. These classes are designed to be lightweight and reflect only the necessary information for each request or response. This abstraction helps to decouple internal domain models from external representations.
- **Repository Layer:** Contains Spring Data JPA repositories that abstract the database operations. Each repository interfaces directly with the database, providing standard CRUD operations and custom queries where needed.

4.2 Security and Cryptography

Security is a fundamental concern of our project, as sensitive operations like checkout and access to personal transaction data must be performed securely and with strong guarantees of authenticity.

To ensure end-to-end data authenticity and integrity, we rely on **digital signatures using Elliptic Curve Cryptography (ECC)**. Each user generates an EC and RSA key pair on their device during account creation. The private key is securely stored in the Android KeyStore, ensuring that it cannot be extracted, while the public key is sent to the server and securely stored in the database, associated with the user account.

This asymmetric key mechanism is applied to the following scenarios:

- **Checkout QR Code Generation:** When the user decides to finalize a purchase, the Client App generates a QR code representing the transaction data. Before generating the QR code, this data is digitally signed using the user's EC private key. The Terminal App then scans this QR code and forwards the signed data to the Server. The Server verifies the authenticity of the request by using the previously stored public key. Only if the signature is valid does the Server proceed with validating and completing the transaction.
- **Voucher Access:** To retrieve their available vouchers, the user's request is also signed with the EC private key, and the Server validates the signature using the corresponding public key before responding. This

ensures that only the legitimate owner of the key pair can request voucher information, preventing impersonation or unauthorized access.

- **Transaction History Access:** Similarly, when requesting past transactions, the request is signed by the Client. The Server again uses the public key to verify the request’s authenticity. This guarantees that transaction records are only accessible to the rightful user.

Using ECC provides strong security guarantees while keeping computational overhead low, which is especially important on mobile devices.

4.3 Database Schema

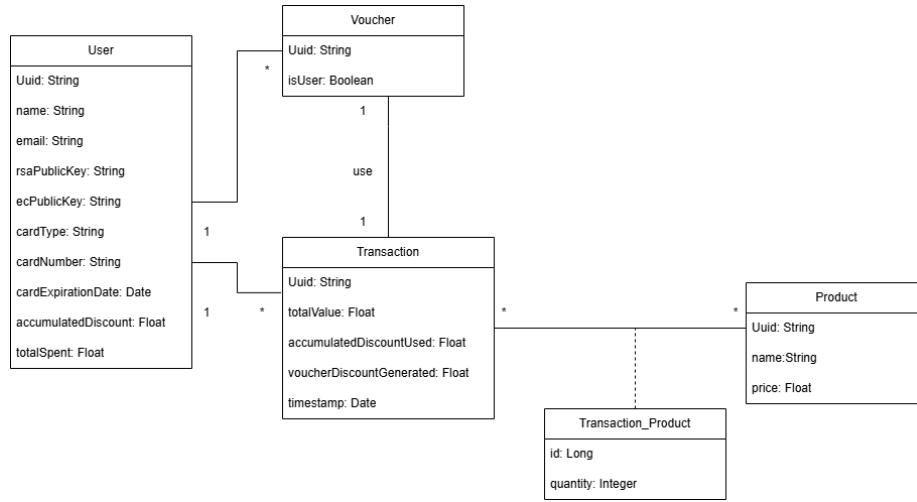


Figure 2: Database Schema

- **User**
Stores user information, including RSA and EC public keys for cryptographic validation, along with data on discounts accumulated through purchases (i.e., unspent discount value), total spending, and card details.
- **Product**
Represents items available for purchase, each with a unique identifier, name, and price.
- **Transaction**
Logs each completed purchase, including total value, accumulated discount used, and the value that the voucher generated. Each transaction is linked to a user and multiple products.

- **Transaction_Product**

A junction table to handle the many-to-many relationship between Transaction and Product, including the quantity of each product bought per transaction.

- **Voucher**

Represents discount vouchers issued to users. Tracks whether a voucher has been used and its association with a specific user and transaction.

4.4 Persistence and Data Integrity

The server application uses H2 as its embedded relational database, configured in **file mode** to ensure persistent storage across sessions. H2 is lightweight, fast, and well-integrated with Spring Boot, making it a suitable choice for academic projects and rapid development environments.

Data access and persistence are managed through JPA (Java Persistence API), with Spring Data JPA simplifying repository definitions and interactions.

The database schema is automatically updated at runtime using Hibernate's `ddl-auto=update` setting.

Additionally, the H2 Console is enabled and accessible via `/h2-console`, which allows us to inspect and query the database directly during development and testing phases.

4.5 API Structure and Communication

The server exposes a RESTful API that supports the full flow of user interaction with the supermarket system (registration, past records, checkout and voucher management). All requests and responses are in the JSON format and communication is secured by digital signatures.

Authentication and User Management

- `POST /api/register`

- Registers a new user and stores their EC and RSA public key.

- `GET /api/customer/userUuid`

- Fetch user data by UUID.

Past Records and Vouchers (Nonce + Signature protected)

- `GET /api/transactions?userUuid=...&nonce=...&signature=...`

- Returns the user's past transactions.

- `GET /api/vouchers?userUuid=...&nonce=...&signature=...`

- Returns the list of unused vouchers.

- GET /api/accumulated-discount?userUuid=...&nonce=...&signature=...
 - Returns the total accumulated discount of the user.

Checkout

- POST /api/checkout
 - Submits a signed transaction for processing and logs the purchase.

Voucher Management

- POST /api/vouchers/create
 - Test endpoint to issue a new voucher to a user.

4.6 Dependency Injection and Modularity

Like the Android apps, the Server takes advantage of Spring's Dependency Injection system to manage components such as services, repositories, and cryptographic helpers.

5 Features & Technologies

Some of the features of the client app are the following:

- **User Authentication:** Users can register in the application by creating an account. Their information is stored both locally and on the server. Once registered, users can log in locally without requiring server access. They can also log out, change their account password, and delete their account.
- **Product Scanning and Shopping Cart Management:** The user can scan product QR codes to add items to the shopping cart. This feature requires access to the device's camera, so it must be used on a physical device. Once items are in the cart, the user has several options: increase the quantity of a specific product, decrease its quantity, remove the product completely, or proceed to checkout.
- **Create a Transaction:** When the user presses the checkout button with the products in the shopping cart, a QR code representing the transaction is generated. This QR code is then scanned by the Terminal Checkout App, which also uses a camera. Transaction information is processed and sent to the server for validation. Completing the checkout clears all items from the shopping cart.

- **Consult Transactions and Vouchers:** The user can view their past transactions and any unused vouchers. This information is stored in the server database and retrieved on demand when requested by the client. The available discounts and vouchers can also be applied before pressing the checkout button.
- **Cryptography:** Various algorithms and techniques are used to ensure secure communication between components, including RSA encryption, ECDSA signatures, and Base64 encoding. Transactions are signed using the user's EC private key and then verified by the server using the user's corresponding EC public key.

To implement the previously described features, a wide range of technologies and libraries were used. Below is a list with a brief explanation of their purpose and how they were applied:

- **Kotlin & Jetpack Compose:** Used for building the Android apps.
- **Kotlin & Spring Boot:** Used for building the backend server.
- **Clean Architecture & MVVM:** Used for structuring the Android apps.
- **Hilt:** Dependency injection library used in the Android apps.
- **Retrofit & OkHttp:** Used for making network requests in the Android apps.
- **Room:** SQLite database used for local storage in the Client App.
- **Jetpack DataStore:** Used for storing user information in the Client App, substituting SharedPreferences.
- **Android Keystore:** Used for storing cryptographic keys securely in the Client App..
- **Zxing:** Library used for QR code scanning in the Android apps.
- **Kotlinx Serialization & Gson:** Used for serializing and deserializing data in the Android apps.
- **KSP:** Kotlin Symbol Processing which substitutes the deprecated kapt.

6 Conclusions

This paper introduces a secure mobile shopping system designed for supermarkets, comprising a Client App, Terminal App, and Server. The system, developed using Kotlin, Jetpack Compose and Spring Boot, offers secure user authentication, QR code scanning for product selection, and transaction history management. Security is prioritized through encryption, digital signatures, and elliptic curve

cryptography (ECC). The architecture follows Clean Architecture principles, with Hilt used for Dependency Injection. The Server component manages user transactions, product information, and voucher handling, with a database supporting various system operations.

References

- [1] Robert Martin. *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Pearson, 2017. <https://www.amazon.com/Clean-Architecture-Craftsmans-Software-Structure/dp/0134494164>.
- [2] Mahach Imangazaliev. Clean architecture manifest. <https://github.com/ImangazalievM/CleanArchitectureManifest>.